INTRODUCTION TO PROGRAMMING

UNIT-5

FUNCTIONS

Definition: A function is a block of code/group of statements/self-contained block of statements/basic building blocks in a program that performs a particular task. It is also known as procedure or subroutine or module, in other programming languages.

To perform any task, we can create function. A function can be called many times. It provides modularity and code reusability.

Advantage of functions

1) Code Reusability

By creating functions in C, you can call it many times. So we don't need to write the same code again and again.

2) Code optimization

It makes the code optimized we don't need to write much code.

3) Easily to debug the program.

Example: Suppose, you have to check 3 numbers (781, 883 and 531) whether it is prime number or not. Without using function, you need to write the prime number logic 3 times. So, there is repetition of code.

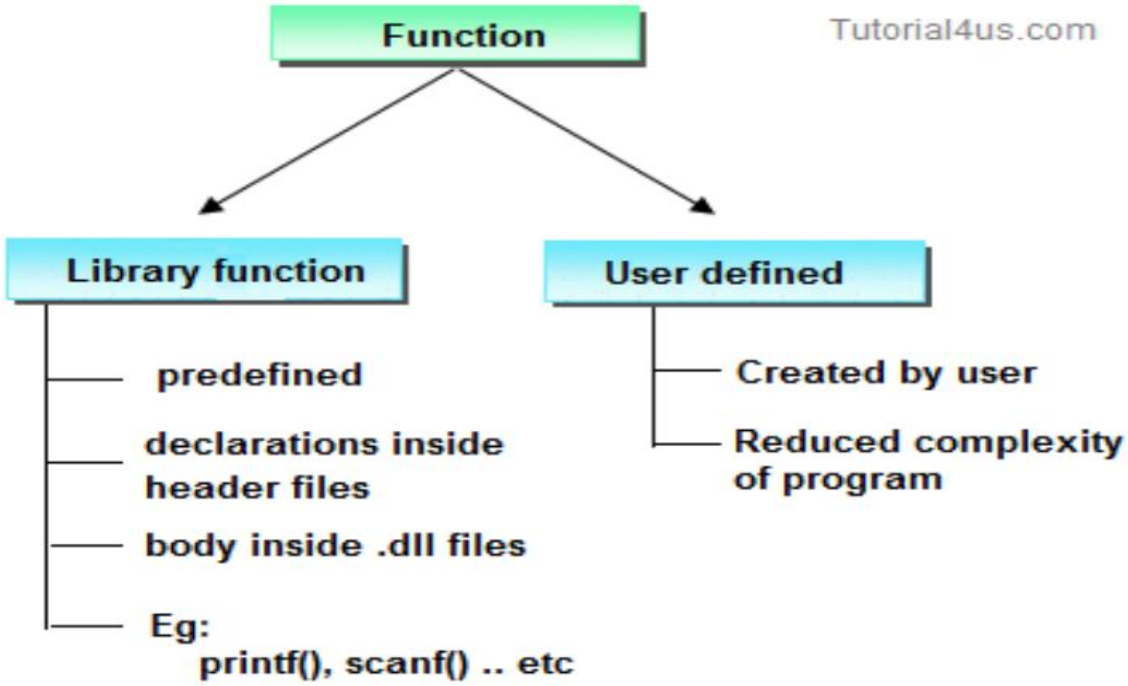
But if you use functions, you need to write the logic only once and you can reuse it several times.

Types of Functions

There are two types of functions in C programming:

1. Library Functions: are the functions which are declared in the C header files such as scanf(), printf(), gets(), puts(), ceil(), floor() etc. You just need to include appropriate header files to use these functions. These are already declared and defined in C libraries. oints to be RememberedSystem defined functions are declared in header filesSystem defined functions are implemented in .dll files. (DLL stands for Dynamic Link Library).To use system defined functions the respective header file must be included.

2. User-defined functions: are the functions which are created by the C programmer, so that he/she can use it many times. It reduces complexity of a big program and optimizes the code. Depending upon the complexity and requirement of the program, you can create as many user-defined functions as you want.



ELEMENTS OF USER-DEFINED FUNCTIONS:

In order to write an efficient user defined function, the programmer must familiar with the following three elements.

1 : Function Declaration. (Function Prototype).

2 : Function Call.

3 : Function Definition

Function Declaration. (Function Prototype).A function declaration is the process of tells the compiler about a function name.

Syntax

return_type function_name(parameter/argument);

return_type function-name();

Ex : int add(int a,int b);

int add();

Note: At the time of function declaration function must be terminated with;

Calling a function/function call

When we call any function control goes to function body and execute entire code.

Syntax : function-name();

function-name(parameter/argument);

return value/ variable = function-name(parameter/argument);

Ex : add(); // function without parameter/argument

add(a,b); // function with parameter/argument

c=fun(a,b); // function with parameter/argument and return values

Defining a function.

Defining of function is nothing but give body of function that means write logic inside function body.

Syntax

return_ type function-name(parameter list) // function header.

{

declaration of variables;

body of function; // Function body

return statement; (expression or value) //optional

}

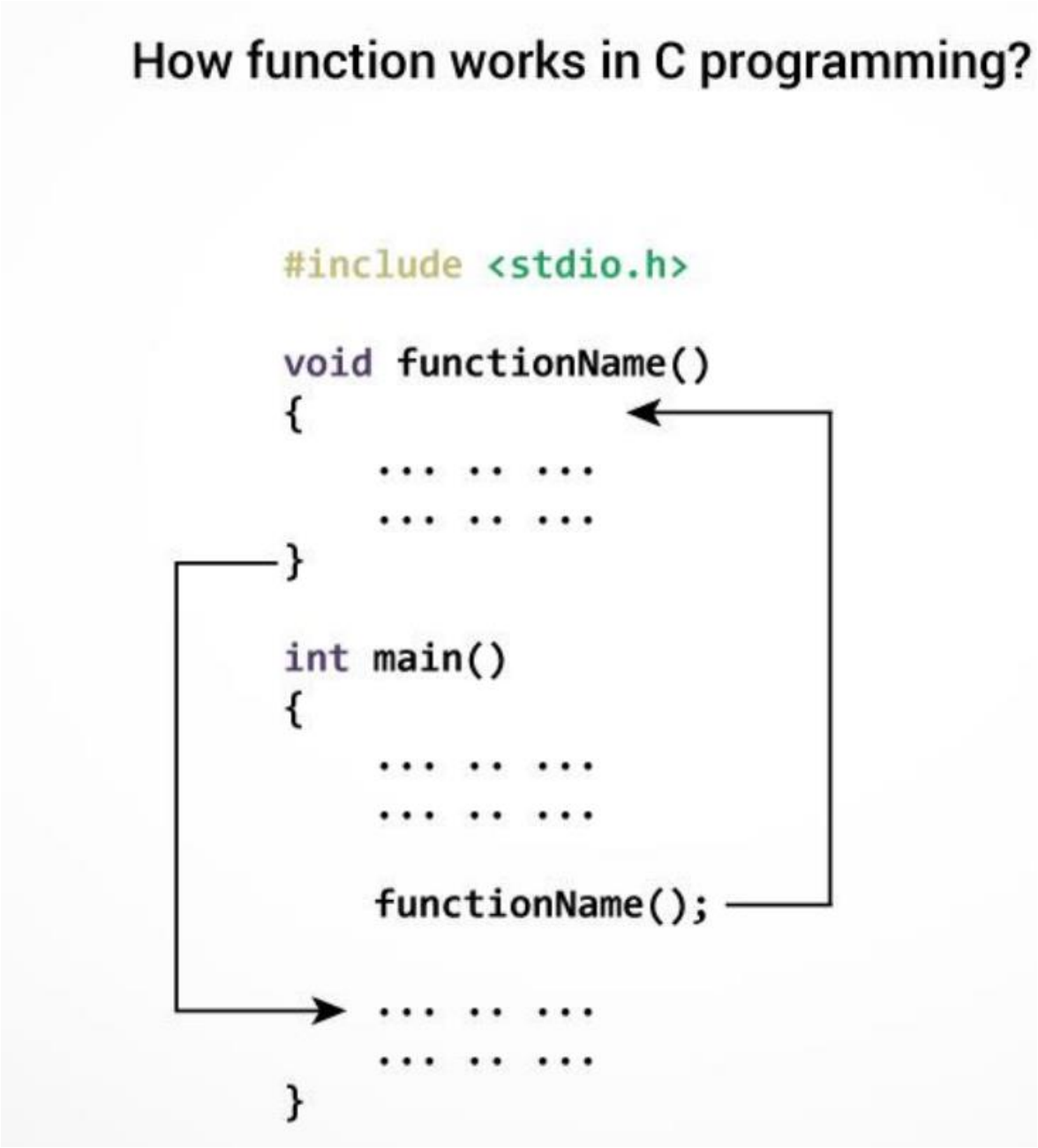
Eg:

```
int  add( int x, int y)
{
        int z;
        z = x + y;

return Z;
    }
```

(or)

```
int add( int x, int y)
    {
    return ( x + y );
    }
```



The execution of a C program begins from the main () function.

When the compiler encounters functionName(); inside the main function, control of the program jumps to void functionName()

And, the compiler starts executing the codes inside the user-defined function.

The control of the program jumps to statement next to functionName(); once all the codes inside the function definition are executed.

Example:

#include<stdio.h>

```
#include<conio.h>
void sum(); // declaring a function
clrscr();
int a=10,b=20, c;
void sum() // defining function
{
c=a+b;
printf("Sum: %d", c);
}
void main()
{
sum(); // calling function
}
```

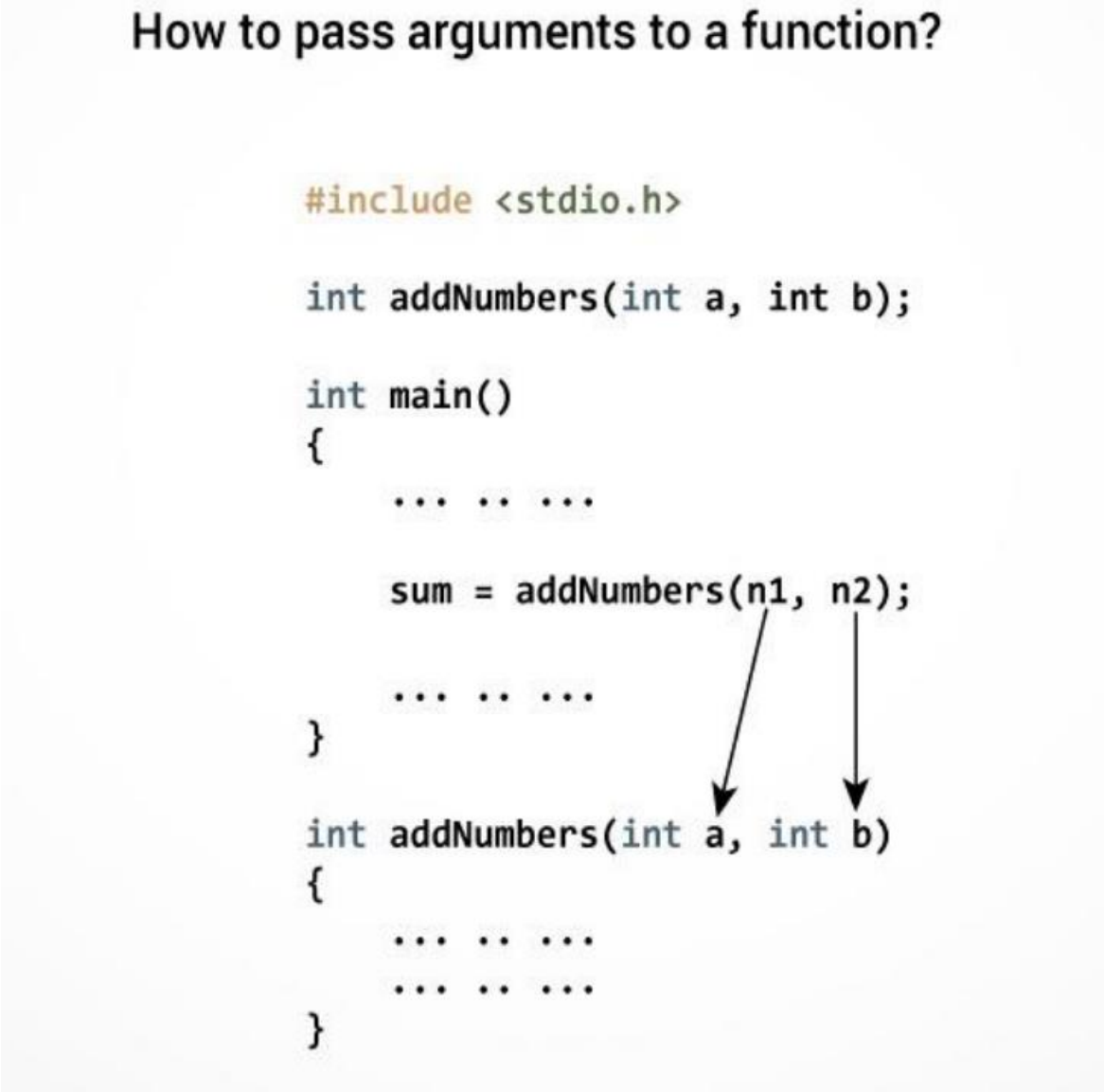
Output

Sum: 30

Example:

```
#include <stdio.h>
int addNumbers(int a, int b); // function prototype
int main()
{
 int n1,n2,sum;
 printf("Enters two numbers: ");
 scanf("%d %d",&n1,&n2);
 sum = addNumbers(n1, n2); // function call
```

```
printf("sum = %d",sum);
 return 0;
}
int addNumbers(int a,int b) // function definition
{
 int result;
 result = a+b;
 return result; // return statement
}
```



Return Statement

Syntax of return statement

Syntax : return; // **does not return any value**

or

return(exp); // **the specified exp value to calling function.**

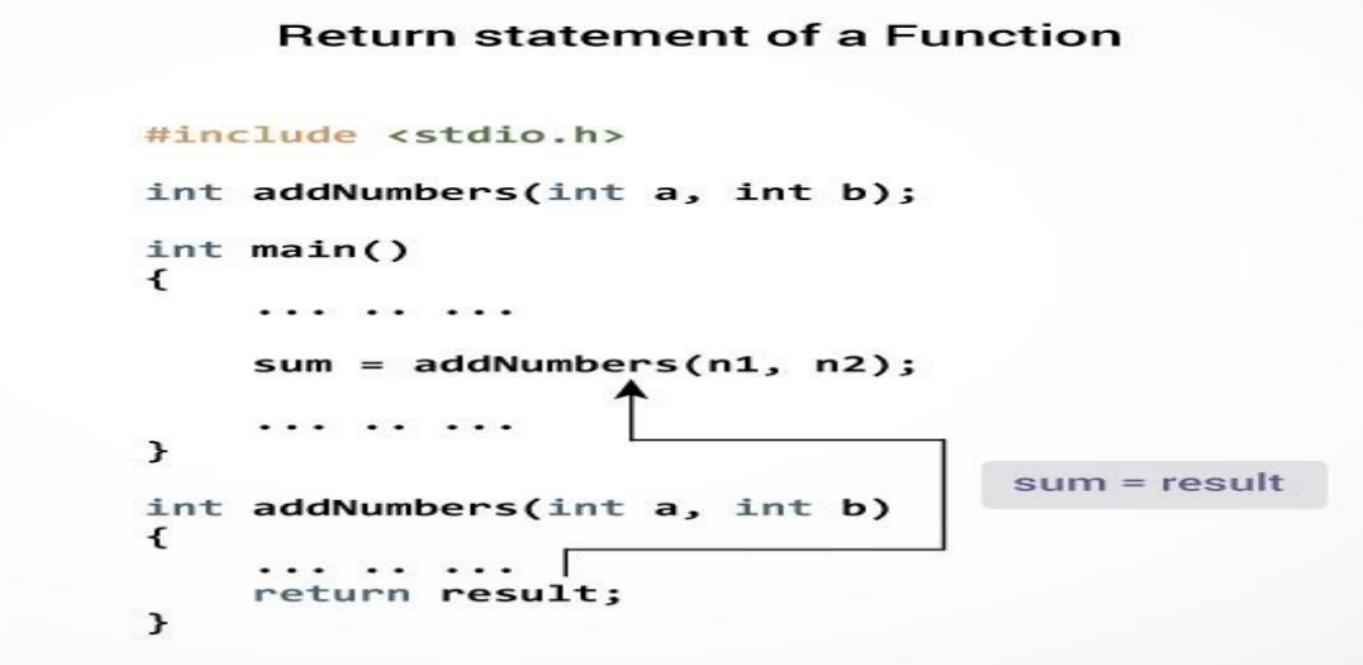
For example,

return a;

return (a+b);

The return statement terminates the execution of a function and returns a value to the calling function. The program control is transferred to the calling function after return statement.

In the above example, the value of variable result is returned to the variable sum in the main() function.



PARAMETERS:

Parameters provides the data communication between the calling function and called function.

They are two types of parameters

1 : Actual parameters.

2 : Formal parameters.

1 : Actual Parameters : These are the parameters transferred from the calling function (main program) to the called function (function).

2 : Formal Parameters :These are the parameters transferred into the calling function (main program) from the called function(function).

The parameters specified in calling function are said to be Actual Parameters.

The parameters declared in called function are said to be Formal Parameters.

The value of actual parameters is always copied into formal parameters.

Ex :

```
main()
 {
 fun1( a , b ); //Calling function
 }
 fun1( x, y ) //called function
 {
 . . . . . .
 }
```

Where

a, b are the Actual Parameters

x, y are the Formal Parameters

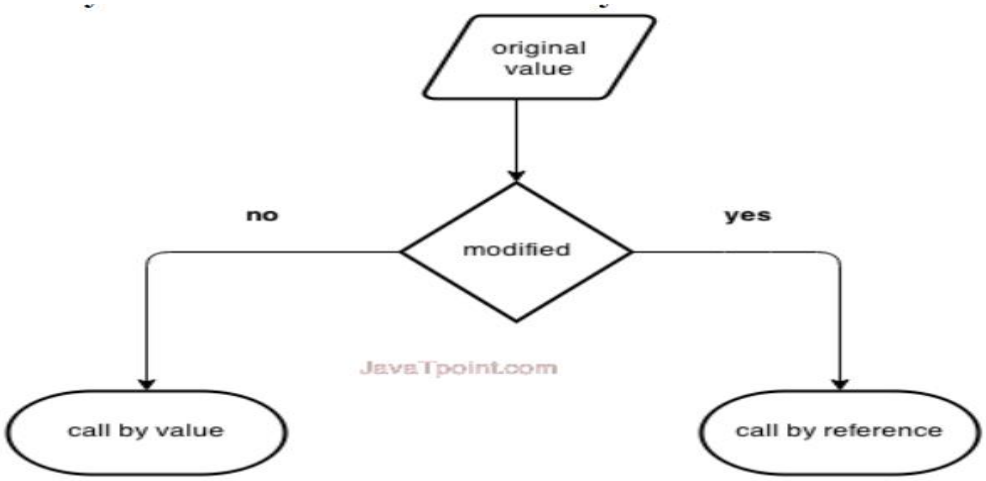
Difference between Actual Parameters and Formal Parameters

Actual Parameters	Formal Parameters
1 : Actual parameters are used in calling function when a function is invoked.	**1 :** Formal parameters are used in the function header of a called function.

Ex : c=add(a,b); Here a,b are actual parameters.	**Ex :** int add(int m,int n); Here m,n are called formal parameters.
2 : Actual parameters can be constants, variables or expression. **Ex :** c=add(a,b) **//variable** c=add(a+5,b); **//expression.** c=add(10,20); **//constants.**	**2 :** Formal parametes should be only variable. Expression and constants are not allowed. **Ex :** int add(int m,n); **//CORRECT** int add(int m+n,int n) **//WRONG** int add(int m,10); **//WRONG**
3 : Actual parameters sends values to the formal parameters. **Ex :** c=add(4,5);	**3 :** Formal parametes receive values from the actual parametes. **Ex :** int add(int m,int n); Here m will have the value 4 and n will have the value 5.
4 : Address of actual parameters can be sent to formal parameters	**4 :** if formal parameters contains address, they should be declared as pointers.

PASSING PARAMETERS TO FUNCTIONS: There are two ways to pass value or data to

Function in C language: call by value and call by reference. Original value is not modified in call by value but it is modified in call by reference.



The called function receives the information from the calling function through the parameters. The variables used while invoking the calling function are called actual parameters and the variables used in the function header of the called function are called formal parameters.

C provides two mechanisms to pass parameters to a function.

1 : Pass by value (OR) Call by value.

2 : Pass by reference (OR) Call by Reference.

1 : Pass by value (OR) Call by value :

In call by value, value being passed to the function is locally stored by the function parameter in stack memory location. If you change the value of function parameter, it is changed for the current function only. It will not change the value of variable inside the caller method such as main ().

Or

When a function is called with actual parameters, the values of actual parameters are copied into formal parameters. If the values of the formal parameters changes in the function, the values of the actual parameters are not changed. This way of passing parameters is called pass by value or call by value.

Ex :

```
#include<stdio.h>
#include<conio.h>
void swap(int ,int );
void main()
{
int i,j;
printf("Enter i and j values:");
scanf("%d%d",&i,&j);
printf("Before swapping:%d%d\n",i,j);
swap(i,j);
```

```
printf("After swapping:%d%d\n",i,j);
getch();
}
void swap(int a,int b)
{
int temp;
temp=a;
a=b;
b=temp;
}
```

Output

Enter i and j values: 10 20

Before swapping: 10 20

After swapping: 10 20

2: Pass by reference (OR) Call by Reference: In pass by reference, a function is called with addresses of actual parameters. In the function header, the formal parameters receive the addresses of actual parameters. Now the formal parameters do not contain values, instead they contain addresses. Any variable if it contains an address, it is called a pointer variable. Using pointer variables, the values of the actual parameters can be changed. This way of passing parameters is called call by reference or pass by reference.

Ex :

```
#include<stdio.h>
#include<conio.h>
void swap(int *,int *);
void main()
{
int i,j;
printf("Enter i and j values:");
```

```
scanf("%d%d",&i,&j);
printf("Before swapping:%d%d\n",i,j);
swap(&i ,&j);
printf("After swapping:%d%d\n",i,j);
}
void swap(int *a,int *b)
{
int temp;
temp=*a;
*a=*b;
*b=temp;
}
```

Output

Enter i and j values: 10 20

Before swapping: 10 20

After swapping: 20 10

Differnce between Call by value and Call by reference

Call by value	**Call by Reference**
1 : When a function is called the values of variables are passed	1 : When a function is called the address of variables are passed.
2 : Change of formal parameters in the function will not affect the actual parameters in the calling function.	**2 :** The actual parameters are changed since the formal parameters indirectly manipulate the actual parametes.
3 : Execution is slower since all the values have to be copied into formal parameters.	**3 :** Execution is faster since only address are copied.

Passing Arrays as Function Arguments

If you want to pass a single-dimension array as an argument in a function, you would have to declare a formal parameter in one of following three ways

Way-1

Formal parameters as a pointer

```
void myFunction(int *param)
{
  .
  .
  .
}
```

Way-2

Formal parameters as a sized array

```
void myFunction(int param[10])
{
  .
  .
  .
}
```

Way-3

Formal parameters as an unsized array

```
void myFunction(int param[])
{
  .
  .
}
```

Example

// Program to calculate the sum of array elements by passing to a function

```
#include <stdio.h>

float calculateSum(float num[]);

int main() {

  float result, num[] = {23.4, 55, 22.6, 3, 40.5, 18};

  // num array is passed to calculateSum()

  result = calculateSum(num);

  printf("Result = %.2f", result);

  return 0;

}

float calculateSum(float num[])

{

  float sum = 0.0;

  for (int i = 0; i < 6; ++i)

 {

    sum += num[i];

  }

  return sum;

}
```

Output

Result = 162.50

To pass an entire array to a function, only the name of the array is passed as an argument.

result = calculateSum(num);

Storage Classes

In C language, each variable has a storage class which is used to define scope and life time of a variable.

Storage: Any variable declared in a program can be stored either in memory or registers. Registers are small amount of storage in CPU. The data stored in registers has fast access compared to data stored in memory.

Storage class of a variable gives information about the location of the variable in which it is stored, initial value of the variable, if storage class is not specified; scope of the variable; life of the variable.

There are four storage classes in C programming.

1 : Automatic Storage class.

2 : Register Storage class.

3 : Static Storage class.

4 : External Storage class.

1: Automatic Storage class: To define a variable as automatic storage class, the keyword „auto" is used. By defining a variable as automatic storage class, it is stored in the memory. The default value of the variable will be garbage value. Scope of the variable is within the block where it is defined and the life of the variable is until the control remains within the block.

Syntax : auto data_type variable_name;

auto int a,b;

Example:

```
void main()
{
int detail;
or
auto int detail; //Both are same
}
```

The variables a and b are declared as integer type and auto. The keyword auto is not mandatory. Because the default storage class in C is auto.

Note: A variable declared inside a function without any storage class specification, is by default an automatic variable. Automatic variables can also be called local variables because they are local to a function.

Ex :

```
void function1();
void function2();
void main()
{
    int x=100;
    function2();
    printf("%d",x);
 }
 void function1()
{
  int x=10;
  printf("%d",x);
}
void function2()
{
  int x=0;
 function1();
printf("%d",x);
}
```

OUTPUT

10

0

100

2: Register Storage class : To define a variable as register storage class, the keyword „register" is used. If CPU cannot store the variables in CPU registers, then the variables are assumed as auto and stored in the memory. When a variable is declared as register, it is stored in the CPU registers. The default value of the variable will be garbage value. Scope of the variable within the block where it is defined and the life of the variables is until the control remains within the block.

Register variable has faster access than normal variable. Frequently used variables are kept in register. Only few variables can be placed inside register.

NOTE : We can't get the address of register variable

Sytax : register data_type variable_name;

Ex: register int i;

Ex :

```
void demo();
void main()
{
   demo();
   demo();
   demo();
}
void demo()
{
   register int i=20;
   printf("%d\n",i);
   i++;
}
```

OUTPUT

20

20

20

3: Static Storage class: When a variable is declared as static, it is stored in the memory. The default value of the variable will be zero. Scope of the variable is within the block where it is defined and the life of the variable persists between different function calls. To define a variable as static storage class, the keyword „static" is used. A static variable can be initialized only once, it cannot be reinitialized.

Syntax : static data_type variable_name;

Ex: static int i;

Ex :

```
void demo();
void main()
{
   demo();
   demo();
   demo();
}
void demo()
{
   static  int i=20;
   printf("%d",i);
   i++;
}
```

OUTPUT

20

21

22

4 : External Storage class : When a variable is declared as extern, it is stored in the memory. The default value is initialized to zero. The scope of the variable is global and the life of the variable is until the program execution comes to an end. To define a variable as external storage class, the keyword „extern" is used. An extern variable is also called as a global variable.

Global variables remain available throughout the entire program. One important thing to remember about global variable is that their values can be changed by any function in the program.

Systax : extern data_type variable_name;

extern int i;

Ex:

```
int number;
void main()
{
number=10;
```

```
}
fun1()
{
number=20;
}
fun2()
{
number=30;
}
```

Here the global variable number is available to all three functions.

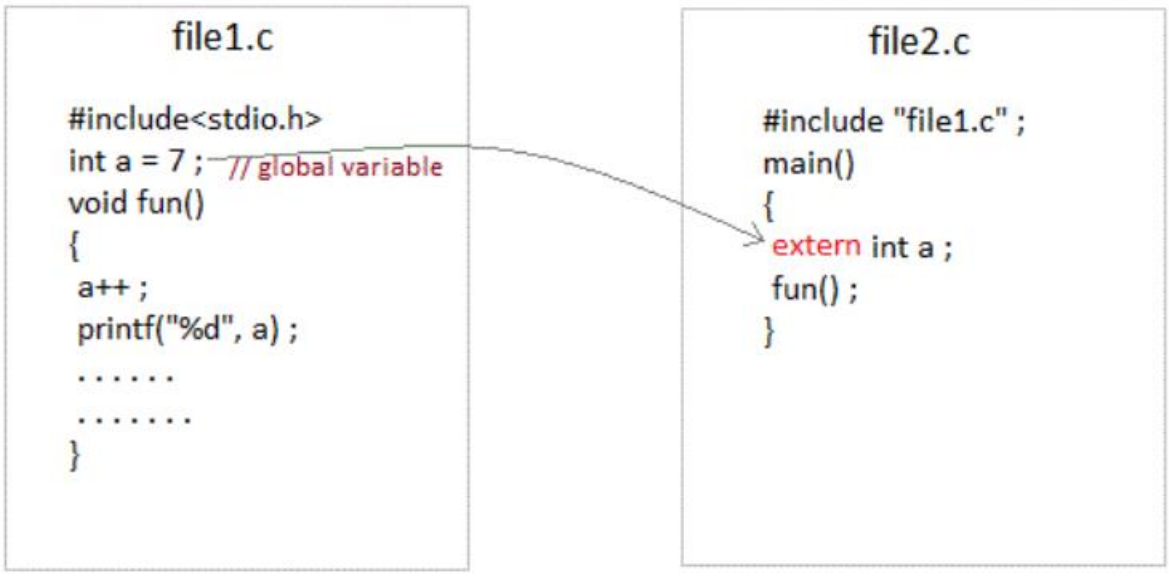
```
Ex : void fun1();
 void fun2();
 int e=20;
 void main()
 {
 fun1();
 fun2();
 }
 void fun1()
{
 extern int e;
 printf(“e number is :%d”,e);
 }
 void fun2()
 {
```

printf(“e number is :%d”,e);

}

extern keyword

The extern keyword is used before a variable to inform the compiler that this variable is declared somewhere else. The extern declaration does not allocate storage for variables.



global variable from one file can be used in other using **extern** keyword.

Storage Classes	Storage Place	Default Value	Scope	Life-time
auto	RAM	Garbage Value	Local	Within function
extern	RAM	Zero	Global	Till the end of main program, May be declared anywhere in the program
static	RAM	Zero	Local	Till the end of main program, Retains value between multiple functions call
register	Register	Garbage Value	Local	Within function

Recursion

When function is called within the same function, it is known as recursion in C. The function which calls the same function, is known as recursive function.

A function that calls itself, and doesn't perform any task after function call, is know as tail recursion. In tail recursion, we generally call the same function with return statement.

Features :

There should be at least one if statement used to terminate recursion.

It does not contain any looping statements.

Advantages :

It is easy to use.

It represents compact programming structures.

Disadvantages :

It is slower than that of looping statements because each time function is called.

Note: while using recursion, programmers need to be careful to define an exit condition from the function, otherwise it will go into an infinite loop. Recursive functions are very useful to solve many mathematical problems, such as calculating the factorial of a number, generating Fibonacci series, etc.

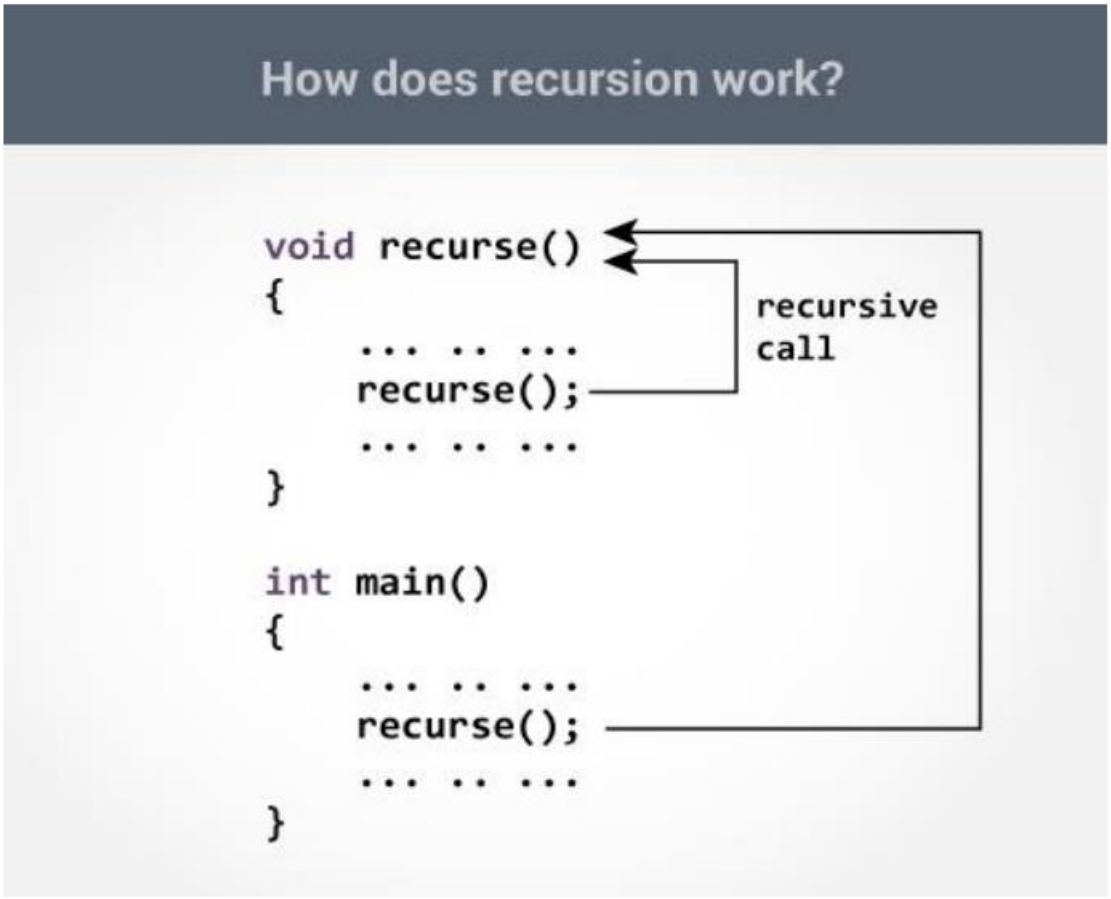
Example of recursion.

```
recursionfunction()

{

recursionfunction();//calling self function

}
```




```
// print factorial number using tail recursion
#include<stdio.h>
#include<conio.h>
int factorial (int n)
{
if ( n < 0)
return -1; /*Wrong value*/
if (n == 0)
return 1; /*Terminating condition*/
return (n * factorial (n -1));
}
void main()
```

```
{
int fact=0;
clrscr();
fact=factorial(5);
printf("\n factorial of 5 is %d",fact);
getch();
}
```

Output

factorial of 5 is 120